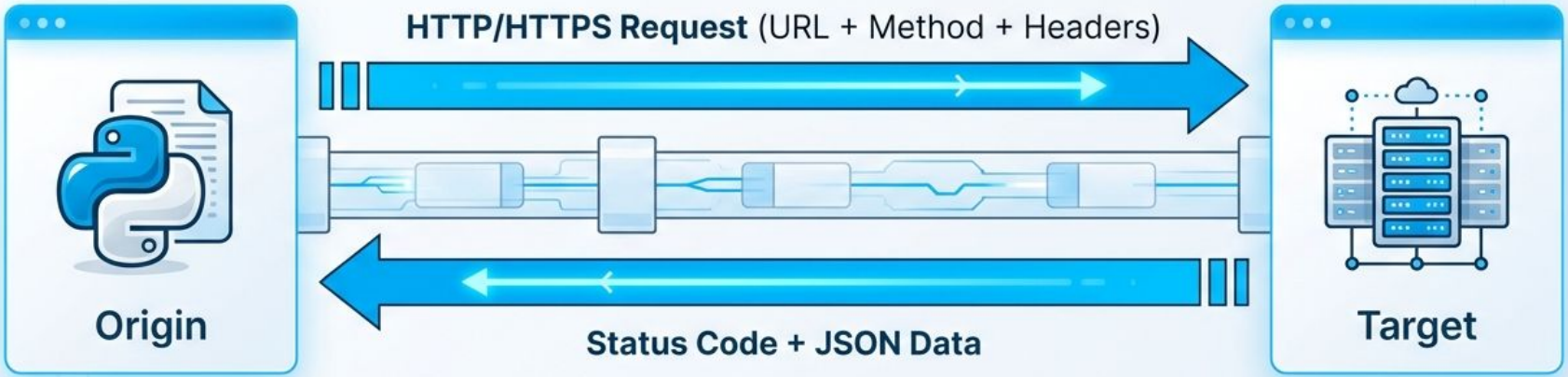
The Building Blocks of DevOps Automation

From manual tasks to Infrastructure-as-Code.



Systems talking to systems



Automates repetitive tasks & retrieves infrastructure info.

Connects disparate tools (e.g., GitLab → API → AWS).

The API Rosetta Stone

HTTP Methods (The Actions)	
GET	Give me data (Example: Get EC2 instances)
POST	Create something (Example: Create Jira ticket)
PUT	Update everything (Example: Update resource)
PATCH	Update something (Example: Change status)
DELETE	Remove something (Example: Delete a record)

Status Codes (The Outcomes)	
200 / 201 / 204	Success (OK / Created / No Content)
400 / 401 / 403	Client Error (Bad Request / Unauthorized / Forbidden)
404	Not Found
500 / 503	Server Error (Internal Server / Service Unavailable)

DevOps Rule: Always check the status code before processing an API response.

Translating Python into API requests



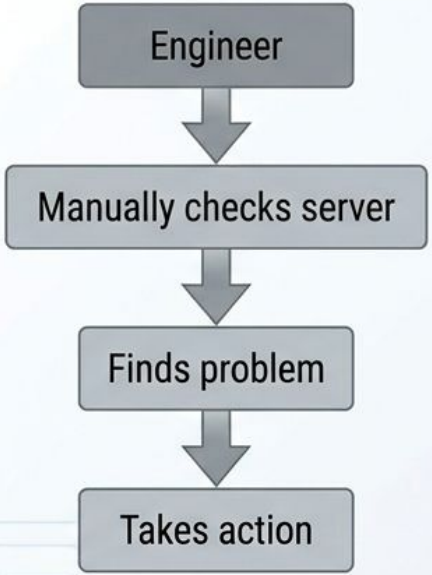
Extracting the signal from the noise

```
192.168.1.10 - - [26/Aug/2026] 'GET /index.html HTTP/1.1' 200
2026-08-26 10:15:23 ERROR Database connection failed
Aug 26 10:15:23 server01 sshd: Failed password
```

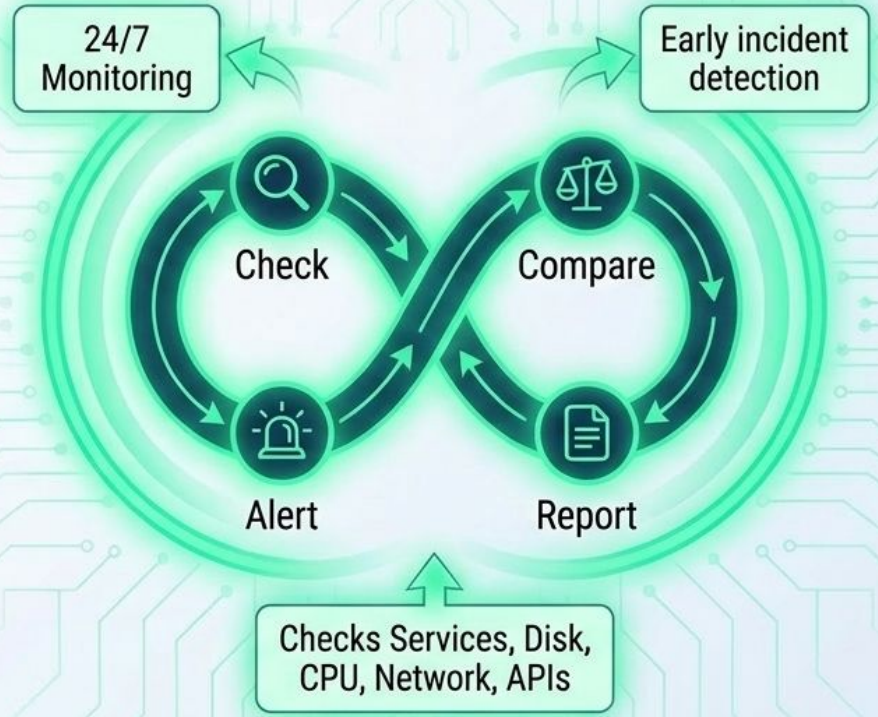


[Date: 2026-08-26] [Time: 10:15:23] [Level: ERROR] [Message: Database connection failed]

The watchful eye of the system



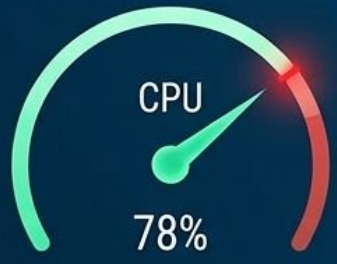
Slow, reactive, and prone to human error.



Evaluating system health in real-time



Disk Usage > 80% → WARNING
(Output: WARNING: Disk usage is 91%)



CPU Usage > 80%
(Checked via top)

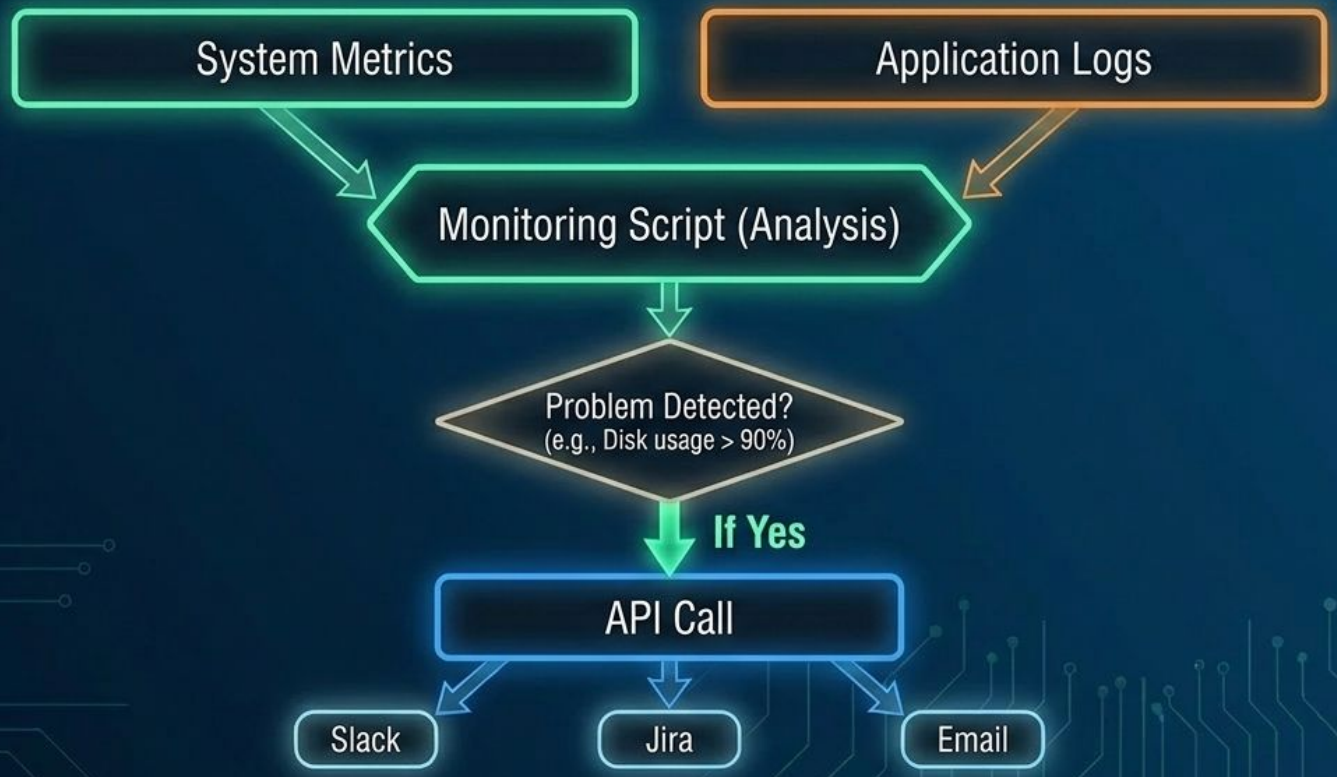


Process: Nginx is running
(Checked via pgrep nginx)

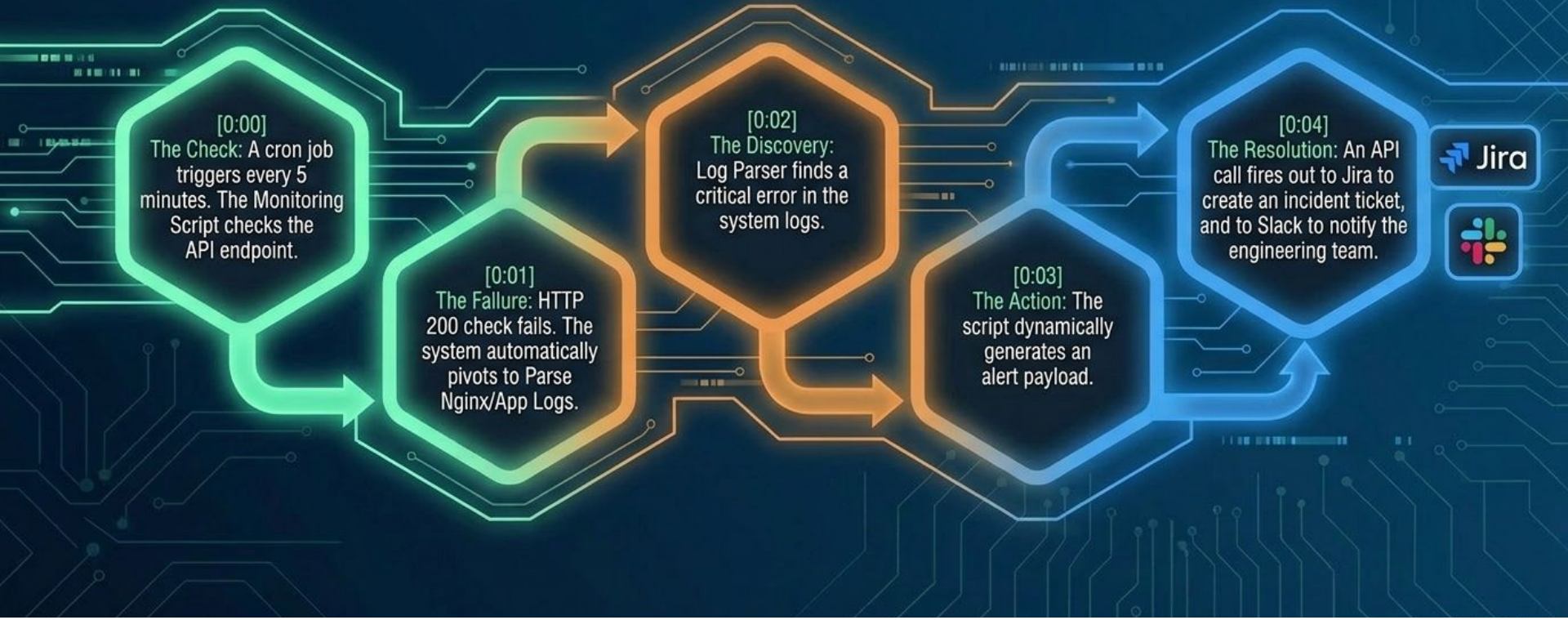
```bash
#!/bin/bash
DISK=$(df / | awk 'NR==2 {print $5}' | tr -d '%')

if [ '$DISK' -gt 80 ]; then
    echo 'WARNING: Disk usage is ${DISK}%'
fi
```

The convergence of DevOps pillars



A fully automated incident response architecture



Rules of engagement for automated systems

API Integrations

- Never expose secrets; utilize environment variables.
- Always set request timeouts.
- Validate responses and handle HTTP errors gracefully.

Log Parsing

- Use structured logs (like JSON) where possible.
- Handle log rotation to avoid processing unnecessarily large files.
- Centralize logs for production systems.

Monitoring Scripts

- Define meaningful thresholds to avoid excessive alert fatigue.
- Make scripts idempotent (safe to run multiple times) where applicable.
- Monitor both infrastructure and application layers.

The anatomy of autonomous infrastructure

APIs

The connective tissue linking disparate applications.

Log Parsing

The analytical engine converting raw text to insight.

Monitoring

The automated sentinels checking system health.

Step 1: Detection (Monitoring)

Step 2: Analysis (Log Parsing)

Step 3: Alerting (API Calls)

Step 4: Incident Response (Automated Resolution)

DevOps automation is the orchestration of these components to replace manual intervention.